

Phase 10: Shell Sort and Final Algorithm Selection — Full Structured Beginner Note

1. Current Progress Before Phase 10

Before Phase 10, you already studied almost the full sorting curriculum.

Phase 1: Sorting Foundations

In Phase 1, you learned the basic language used to understand sorting algorithms.

Term	Simple meaning
Sorting	Arranging data in order
Ascending order	Smallest to largest
Descending order	Largest to smallest
Comparison	Checking values to decide their order
Swap	Exchanging two values
Shift	Moving values to make space
Adaptive	Faster when data is already sorted or nearly sorted
Extra memory	Additional space used by an algorithm
Stable	Equal values keep their original relative order
Big O	Describes how algorithm work grows
$O(n)$	Work grows directly with input size
$O(n^2)$	Work grows very quickly, often because of nested loops
$O(n \log n)$	Usually a fast comparison-based sorting class
$O(1)$ space	Uses only a fixed small amount of extra memory

The most important lesson from Phase 1 was:

Do not judge a sorting algorithm only by asking, "Does it sort?"

A better question is:

How does it sort, how much time does it take, how much memory does it use, is it stable, and is it adaptive?

Phase 2: Bubble Sort

Bubble Sort compares neighboring values and swaps them if they are in the wrong order.

Example:

8 5

Since:

8 > 5

Bubble Sort swaps them:

5 8

Important points from Bubble Sort:

- It compares adjacent elements.
- It uses swaps.
- One full scan is called a pass.
- After each pass, one large value is fixed at the end.
- After k passes, the k largest values are fixed at the end.
- It can be adaptive if we use a flag.
- It is stable if we use $>$ instead of $>=$.
- Best-case time is $O(n)$ with the adaptive flag.
- Worst-case time is $O(n^2)$.
- Space complexity is $O(1)$.

Phase 3: Insertion Sort

Insertion Sort builds a sorted left part.

Example:

[5 8] 7 3 2

The left part [5 8] is already sorted.

Now we insert 7 into the correct position:

[5 7 8] 3 2

Important points from Insertion Sort:

- It inserts one value into a sorted left part.
- It starts from index `1` because the first element alone is already sorted.
- It stores the selected value in `x` or `temp`.
- It shifts bigger values to the right.
- It is naturally adaptive.
- It is stable if we use `>` instead of `>=`.
- Best-case time is $O(n)$.
- Worst-case time is $O(n^2)$.
- Space complexity is $O(1)$.
- It is good for small or nearly sorted arrays.

Phase 4: Selection Sort

Selection Sort selects the smallest value from the unsorted part and puts it at the front.

Example:

8 6 3 2 5 4

The smallest value is `2`, so Selection Sort swaps it with the first value:

2 6 3 8 5 4

Important points from Selection Sort:

- It selects the minimum value.
- It fills the array from left to right.
- It uses `i`, `j`, and `k`.
- `i` is the position being filled.
- `j` scans the unsorted part.
- `k` stores the index of the smallest value found so far.
- It swaps only after the inner scan finishes.
- It performs fewer swaps than Bubble Sort.
- It is not adaptive.
- It is usually not stable.
- Best-case time is $O(n^2)$.
- Worst-case time is $O(n^2)$.
- Space complexity is $O(1)$.

- After `k` passes, the `k` smallest values are fixed at the front.
-

Phase 5 and Phase 6: Quick Sort

Quick Sort introduced two main ideas:

1. Partitioning
2. Recursion

Partitioning means:

Choose one value, called the pivot, and place it in its final sorted position.

Example:

```
50 70 60 90 40 80 10 20 30 ∞
```

The pivot is `50`.

After partitioning:

```
40 30 20 10 | 50 | 80 90 60 70
```

Now `50` is in its final sorted position.

But the left side is not fully sorted:

```
40 30 20 10
```

And the right side is not fully sorted:

```
80 90 60 70
```

So Quick Sort uses recursion:

```
Partition the array.  
Sort the left side recursively.  
Sort the right side recursively.
```

Simple memory trick:

Partition, sort left, sort right.

Important points from Quick Sort:

- Quick Sort is a divide-and-conquer algorithm.
- It chooses a pivot.
- Partitioning places the pivot in its correct position.
- Recursion sorts the smaller parts.
- Best-case time is $O(n \log n)$.
- Average-case time is $O(n \log n)$.
- Worst-case time is $O(n^2)$.
- First-element pivot Quick Sort can be bad for already sorted data.
- It is usually not stable.
- It mostly sorts inside the original array, but recursion uses stack memory.

Phase 7 and Phase 8: Merge Sort

Merge Sort introduced merging.

Merging means:

Combine two already sorted lists into one bigger sorted list.

Example:

A = [2, 6, 10]
B = [3, 7]

Both lists are already sorted.

After merging:

C = [2, 3, 6, 7, 10]

Important points from Merge Sort:

- Merging works only when the two input parts are already sorted.
- Merging uses i , j , and k .
- i reads the left section.
- j reads the right section.
- k writes into the temporary array.
- A temporary array is used to safely store the merged result.

- Leftover values must be copied after one side finishes.
- The merged values must be copied back into the original array.
- Iterative Merge Sort uses loops.
- Recursive Merge Sort uses recursion.
- Merge Sort has $O(n \log n)$ time complexity.
- Merge Sort uses extra memory.
- Merge Sort can be stable when equal values are handled correctly.

Phase 9: Index-Based Sorting

Phase 9 introduced a different sorting family.

Before Phase 9, most algorithms were comparison-based.

Comparison-based sorting asks questions like:

Is $A[i]$ greater than $A[j]$?
 Is $A[j]$ greater than $A[j + 1]$?
 Is this value smaller than the pivot?

Phase 9 algorithms use values as:

- indexes
- positions
- bins
- digits

The three main algorithms were:

Algorithm	Main idea
Count Sort	Count how many times each value appears
Bin Sort	Put values into linked-list bins
Radix Sort	Sort numbers digit by digit

Important points from Phase 9:

- Index-based sorting can be very fast in suitable cases.
- Count Sort can be fast when the value range is small.
- Bin Sort uses linked-list bins.
- Radix Sort uses digit-by-digit sorting.
- These algorithms usually need extra memory.
- They are not always suitable for all kinds of data.

2. Why Phase 10 Comes After Phase 9

Phase 10 is the final phase of the curriculum.

By now, you have learned:

- simple comparison sorts
- divide-and-conquer sorts
- index-based sorts

Now Phase 10 returns to a familiar idea:

Insertion Sort

But it improves it.

Insertion Sort is good for nearly sorted arrays, but it can be slow when small values are far away from their correct positions.

Shell Sort improves Insertion Sort by using gaps.

So Phase 10 teaches:

Shell Sort = Insertion Sort with gaps

It also gives final guidance for choosing the correct sorting algorithm in different situations.

3. Main Goal of Phase 10

The main goal of Phase 10 is to understand Shell Sort.

In simple words:

Shell Sort improves Insertion Sort by allowing elements to move long distances first, then shorter distances, and finally one position at a time.

Phase 10 covers:

- What Shell Sort is
- Why Shell Sort improves Insertion Sort
- What a gap is

- Gap sequence
- Gap-based insertion logic
- Shell Sort dry run
- Shell Sort C program
- Code explanation
- Time complexity
- Space complexity
- Adaptiveness
- Stability
- Common beginner mistakes
- Final algorithm selection guide

Simple memory trick:

Big gaps first, small gaps later, gap 1 finishes.

4. What Is Shell Sort?

Shell Sort is a sorting algorithm based on Insertion Sort.

But instead of comparing only neighboring elements, Shell Sort compares elements that are far apart.

This distance is called a gap.

Normal Insertion Sort moves values like this:

one step at a time

Shell Sort moves values like this:

many steps first, then fewer steps later

That is why Shell Sort can be faster than ordinary Insertion Sort.

5. Weakness of Insertion Sort

Insertion Sort is very good when the array is already nearly sorted.

But it has one weakness:

If a small value is far on the right side, Insertion Sort moves it left slowly.

Example:

```
11 13 7 12 16 9 24 5 10 3
```

The value **3** is very small, but it is at the far right.

In normal Insertion Sort, **3** must move left one position at a time:

```
3 moves past 10
3 moves past 5
3 moves past 24
3 moves past 9
3 moves past 16
...
```

This takes many shifts.

Shell Sort solves this by allowing values to jump over larger gaps first.

6. Main Idea of Shell Sort

Shell Sort says:

Do not only compare neighboring values. First compare values that are far apart.

Example:

Instead of comparing:

```
A[0] with A[1]
A[1] with A[2]
A[2] with A[3]
```

Shell Sort may first compare:

```
A[0] with A[5]  
A[1] with A[6]  
A[2] with A[7]
```

This allows small values to move closer to the beginning quickly.

After that, the gap becomes smaller.

Finally, when the gap becomes `1`, Shell Sort behaves like normal Insertion Sort.

But at that point, the array is already nearly sorted.

7. What Is a Gap?

A gap is the distance between two elements being compared.

Gap in normal Insertion Sort

In normal Insertion Sort, the gap is always `1`.

Example:

```
A[j] and A[j + 1]
```

This means values are next to each other.

So normal Insertion Sort moves values one position at a time.

Gap in Shell Sort

In Shell Sort, the gap can be larger than `1`.

Example:

```
gap = 5
```

This means we compare elements that are 5 positions apart.

Example:

A[0] and A[5]
A[1] and A[6]
A[2] and A[7]
A[3] and A[8]
A[4] and A[9]

So values can move faster.

8. Gap Sequence

A gap sequence is the list of gaps used by Shell Sort.

In this curriculum, the gap starts as:

$\text{gap} = n / 2$

Then the gap is divided by 2 each time:

$\text{gap} = \text{gap} / 2$

This continues until the gap becomes 1.

Example:

If:

$n = 10$

Then:

$\text{gap} = 10 / 2 = 5$
 $\text{gap} = 5 / 2 = 2$
 $\text{gap} = 2 / 2 = 1$

So the gap sequence is:

$5 \rightarrow 2 \rightarrow 1$

Another way to write the pattern is:

$n/2, n/4, \dots, 1$

Important rule:

The final gap must be 1.

Why?

Because when `gap = 1`, Shell Sort performs a normal Insertion Sort pass and fully finishes the sorting.

9. Beginner Analogy

Imagine students are sitting in a row, but they need to be arranged by height.

Insertion Sort method

Insertion Sort says:

Move a short student left one seat at a time.

This works, but it can be slow.

Shell Sort method

Shell Sort says:

First let the short student jump many seats closer to the correct place. Later, make small corrections.

So Shell Sort works like this:

Big jumps first.
Small jumps later.
Final normal insertion pass.

Memory trick:

Shell Sort = Insertion Sort with gaps.

10. How Shell Sort Works Step by Step

Shell Sort works like this:

1. Start with a large gap.
2. Compare elements that are gap distance apart.
3. Use insertion-style shifting among those gap elements.
4. Move bigger values forward by gap distance.
5. Insert the selected value into its correct gap-position.
6. Reduce the gap.
7. Repeat until the gap becomes `1`.
8. The final gap-1 pass completes the sorting.

Simple version:

```
gap = n / 2
perform insertion-style sorting using this gap

gap = gap / 2
perform insertion-style sorting using this gap

continue until gap = 1
```

11. Shell Sort Compared With Insertion Sort

Shell Sort is very similar to Insertion Sort, but with one important change:

Instead of moving by 1 position, it moves by gap positions.

Insertion Sort	Shell Sort
Moves values one position at a time	Moves values using large gaps first
Uses <code>j--</code>	Uses <code>j = j - gap</code>
Writes into <code>A[j + 1]</code>	Writes into <code>A[j + gap]</code>
Good for nearly sorted data	Makes the data nearly sorted first
Can be slow if small values are far away	Handles far-away values better
Final logic is simple insertion	Final <code>gap = 1</code> pass becomes insertion sort

Important formula difference:

Normal Insertion Sort:

```
j--;  
A[j + 1] = A[j];
```

Shell Sort:

```
j = j - gap;  
A[j + gap] = A[j];
```

12. Important Variables in Shell Sort

Variable	Simple meaning
A[]	The array being sorted
n	Number of elements in the array
gap	Distance between compared elements
i	Scans forward through the array
j	Moves backward by gap distance
temp	Temporarily stores the value being inserted

13. Shell Sort Pseudocode

Simple pseudocode:

```
ShellSort(A, n):  
    for gap = n/2; gap >= 1; gap = gap/2:  
        for i = gap to n - 1:  
            temp = A[i]  
            j = i - gap  
  
            while j >= 0 and A[j] > temp:  
                A[j + gap] = A[j]  
                j = j - gap  
  
            A[j + gap] = temp
```

Simple explanation:

- Start with a large gap.
 - Pick one value as `temp`.
 - Look backward by gap distance.
 - Shift bigger gap-values forward.
 - Insert `temp` in the correct place.
 - Reduce the gap and repeat.
-

14. Shell Sort C Program

```
#include <stdio.h>

void ShellSort(int A[], int n) {
    int gap, i, j, temp;

    for (gap = n / 2; gap >= 1; gap /= 2) {
        for (i = gap; i < n; i++) {
            temp = A[i];
            j = i - gap;

            while (j >= 0 && A[j] > temp) {
                A[j + gap] = A[j];
                j = j - gap;
            }

            A[j + gap] = temp;
        }
    }
}

int main() {
    int A[] = {11, 13, 7, 12, 16, 9, 24, 5, 10, 3};
    int n = sizeof(A) / sizeof(A[0]);

    ShellSort(A, n);

    for (int i = 0; i < n; i++) {
        printf("%d ", A[i]);
    }

    printf("\n");
    return 0;
}
```

Expected output:

```
3 5 7 9 10 11 12 13 16 24
```

15. Explanation of the Code

The program has two main parts:

1. `ShellSort()` function
2. `main()` function

16. The `ShellSort()` Function

Function header:

```
void ShellSort(int A[], int n)
```

This function receives:

Parameter	Meaning
<code>A[]</code>	The array to sort
<code>n</code>	Number of elements in the array

17. Declaring Variables

```
int gap, i, j, temp;
```

Meaning:

Variable	Meaning
<code>gap</code>	Controls the distance between compared elements
<code>i</code>	Selects the current value to insert
<code>j</code>	Moves backward by gap distance

Variable	Meaning
temp	Saves the selected value

18. Outer Loop: Controlling the Gap

```
for (gap = n / 2; gap >= 1; gap /= 2)
```

This loop controls the gap sequence.

If:

```
n = 10
```

Then the gaps are:

```
5 → 2 → 1
```

Meaning:

- When `gap = 5`, elements can move far.
- When `gap = 2`, the array becomes more refined.
- When `gap = 1`, normal Insertion Sort finishes the job.

Important:

The loop must reach `gap = 1`.

Without the gap-1 pass, the array may not be fully sorted.

19. Inner Loop: Scanning the Array

```
for (i = gap; i < n; i++)
```

This loop scans the array starting from index `gap`.

Why start from `gap`?

Because the first element that has a previous gap element is at index `gap`.

Example:

If:

```
gap = 5
```

Then:

```
i starts at 5
```

Because index `5` can be compared with index `0`.

20. Saving the Current Value

```
temp = A[i];
```

This saves the current value before shifting begins.

Why is this necessary?

Because shifting values may overwrite `A[i]`.

This is the same reason we save `x` in Insertion Sort.

Simple meaning:

```
Keep the value safe before moving other values.
```

21. Setting `j`

```
j = i - gap;
```

This means:

Start checking from the previous element in the same gap group.

Example:

If:

```
i = 5  
gap = 5
```

Then:

```
j = 5 - 5 = 0
```

So Shell Sort compares:

```
A[5] with A[0]
```

22. Shifting Bigger Values

```
while (j >= 0 && A[j] > temp) {  
    A[j + gap] = A[j];  
    j = j - gap;  
}
```

This is the heart of Shell Sort.

The condition:

```
j >= 0 && A[j] > temp
```

means:

1. `j >= 0`: stay inside the array.
2. `A[j] > temp`: the previous gap-value is bigger than the value we want to insert.

If both are true, shift the bigger value forward:

```
A[j + gap] = A[j];
```

Then move backward again by gap distance:

```
j = j - gap;
```

Simple meaning:

While bigger values are found, shift them forward by gap distance.

23. Inserting `temp`

```
A[j + gap] = temp;
```

When the loop stops, the correct position for `temp` is:

```
j + gap
```

So we insert `temp` there.

This is similar to Insertion Sort, where we insert using:

```
A[j + 1] = x;
```

But in Shell Sort, we use:

```
A[j + gap] = temp;
```

24. The `main()` Function

```
int main() {  
    int A[] = {11, 13, 7, 12, 16, 9, 24, 5, 10, 3};  
    int n = sizeof(A) / sizeof(A[0]);  
  
    ShellSort(A, n);  
  
    for (int i = 0; i < n; i++) {  
        printf("%d ", A[i]);  
    }  
}
```

```
    printf("\n");  
    return 0;  
}
```

This function:

1. Creates the array.
2. Calculates the number of elements.
3. Calls `ShellSort(A, n)`.
4. Prints the sorted array.

The line:

```
int n = sizeof(A) / sizeof(A[0]);
```

means:

number of elements = total array size / size of one element

25. Dry Run of Shell Sort

Let us sort this array:

11 13 7 12 16 9 24 5 10 3

Final sorted result should be:

3 5 7 9 10 11 12 13 16 24

Number of elements:

n = 10

Gap sequence:

5 → 2 → 1

26. Gap = 5

When:

```
gap = 5
```

Shell Sort compares elements 5 positions apart.

Initial array:

```
11 13 7 12 16 9 24 5 10 3
```

Pairs compared:

```
Index 0 and 5: 11 and 9  
Index 1 and 6: 13 and 24  
Index 2 and 7: 7 and 5  
Index 3 and 8: 12 and 10  
Index 4 and 9: 16 and 3
```

Now small values can jump far left.

Pair 1: Compare 11 and 9

```
11 > 9
```

So **9** moves before **11**.

Pair 2: Compare 13 and 24

```
13 > 24
```

This is false, so no change.

Pair 3: Compare 7 and 5

```
7 > 5
```

So **5** moves before **7**.

Pair 4: Compare 12 and 10

12 > 10

So 10 moves before 12.

Pair 5: Compare 16 and 3

16 > 3

So 3 moves before 16.

After the gap-5 pass:

9 13 5 10 3 11 24 7 12 16

Important:

This array is not fully sorted yet.

But small values like 3, 5, and 9 moved closer to the left.

That is the purpose of large gaps.

27. Gap = 2

Now the gap becomes:

gap = 2

Current array:

9 13 5 10 3 11 24 7 12 16

Now Shell Sort compares and inserts values 2 positions apart.

This means it works on gap groups such as:

Indexes 0, 2, 4, 6, 8
Indexes 1, 3, 5, 7, 9

Group 1: Indexes 0, 2, 4, 6, 8

Values:

9 5 3 24 12

After insertion-style sorting within this group:

3 5 9 12 24

Group 2: Indexes 1, 3, 5, 7, 9

Values:

13 10 11 7 16

After insertion-style sorting within this group:

7 10 11 13 16

After the gap-2 pass, the full array becomes:

3 7 5 10 9 11 12 13 24 16

It is still not fully sorted, but it is much closer.

28. Gap = 1

Now the gap becomes:

gap = 1

Current array:


```
3 7 5 10 9 11 12 13 24 16
```

When `gap = 1`, Shell Sort becomes normal Insertion Sort.

But the array is already almost sorted, so this final pass is easy.

Final sorted array:

```
3 5 7 9 10 11 12 13 16 24
```

29. Dry Run Summary

Original array:

```
11 13 7 12 16 9 24 5 10 3
```

Gap sequence:

```
5 → 2 → 1
```

After gap `5`:

```
9 13 5 10 3 11 24 7 12 16
```

After gap `2`:

```
3 7 5 10 9 11 12 13 24 16
```

After gap `1`:

```
3 5 7 9 10 11 12 13 16 24
```

Final sorted result:

```
3 5 7 9 10 11 12 13 16 24
```

30. Why Shell Sort Works

Shell Sort works because it slowly improves the array.

Large gaps move badly placed elements closer to their correct area.

Smaller gaps refine the order.

The final gap-1 pass finishes the sorting.

Simple idea:

Large gaps make big corrections.
Small gaps make small corrections.
Gap 1 completes the sort.

Insertion Sort is fast when the array is nearly sorted.

Shell Sort makes the array nearly sorted before the final insertion pass.

That is why Shell Sort improves Insertion Sort.

31. Shell Sort Properties

Property	Shell Sort
Sorting type	Comparison-based sorting
Main idea	Gap-based Insertion Sort
Uses swaps or shifts?	Mainly shifts, like Insertion Sort
In-place?	Yes
Extra space	$O(1)$
Adaptive?	It benefits from nearly sorted data
Stable?	Usually no
Final gap needed?	Yes, gap must reach 1
Best use	Improving Insertion Sort without extra arrays

32. Is Shell Sort Comparison-Based?

Yes.

Shell Sort compares values using:

```
A[j] > temp
```

This means it is a comparison-based sorting algorithm.

It is not index-based like Count Sort, Bin Sort, or Radix Sort.

33. Is Shell Sort In-Place?

Yes.

Shell Sort sorts inside the original array.

It does not create another array.

It only uses a few variables:

```
gap, i, j, temp
```

So the extra space is:

```
O(1)
```

34. Is Shell Sort Stable?

Usually, Shell Sort is not stable.

A stable sorting algorithm keeps equal values in their original relative order.

Shell Sort can compare and move values over gaps.

Because values can jump over other values, equal records may change their original order.

So Shell Sort is usually considered:

Not stable

Important:

This does not mean Shell Sort gives wrong sorting.

It means equal records may not keep their original order.

35. Is Shell Sort Adaptive?

Shell Sort is based on Insertion Sort logic.

Insertion Sort is adaptive because it works faster when the array is already nearly sorted.

Shell Sort also benefits when the data becomes nearly sorted during earlier gap passes.

So we can say:

Shell Sort has adaptive behavior because it uses insertion-style logic.

But it still performs gap passes according to the gap sequence.

36. Time Complexity of Shell Sort

Shell Sort time complexity is not as simple as Bubble Sort or Selection Sort.

Why?

Because Shell Sort performance depends on the gap sequence.

The curriculum gap sequence is:

$n/2, n/4, \dots, 1$

For beginner understanding, remember:

Shell Sort is usually faster than simple $O(n^2)$ sorts.

But the exact time depends on the gaps used.

Some explanations describe it around:

$O(n \log n)$

Other analyses describe it near:

$O(n^{1.5})$

or:

$O(n^{1.6})$

The safest beginner answer is:

Shell Sort is faster than ordinary Insertion Sort in many cases, but its exact time complexity depends on the gap sequence.

37. Space Complexity of Shell Sort

Shell Sort uses only a few extra variables:

gap, i, j, temp

It does not use:

- extra array
- linked list bins
- counter array
- merge array

So its space complexity is:

$O(1)$

This means Shell Sort is in-place.

38. Why Shell Sort Is Better Than Insertion Sort

Insertion Sort moves elements one position at a time.

Shell Sort moves elements using gaps.

Example:

If a small value is at the far right, Insertion Sort may need many shifts to move it to the left.

Shell Sort can move it closer to the left during an early large-gap pass.

Then later passes only need small adjustments.

So Shell Sort is better because:

- it moves far-away values faster
- it reduces the work needed in the final insertion pass
- it remains in-place
- it uses only $O(1)$ extra space

39. Where Shell Sort Fits in the Curriculum

Shell Sort belongs to the comparison-based sorting family.

Sorting family	Algorithms
Simple comparison sorts	Bubble, Insertion, Selection
Divide-and-conquer comparison sorts	Quick Sort, Merge Sort
Index-based sorts	Count Sort, Bin Sort, Radix Sort
Gap-based comparison sort	Shell Sort

Shell Sort is special because it is not completely new from zero.

It is built from Insertion Sort.

Simple memory:

Shell Sort improves Insertion Sort.

40. Common Beginner Mistakes in Shell Sort

Mistake 1: Thinking gap groups are real separate arrays

Gap groups are not real separate arrays.

They are only logical groups inside the same array.

Example with `gap = 2`:

```
Indexes 0, 2, 4, 6, 8  
Indexes 1, 3, 5, 7, 9
```

These are just positions inside the same original array.

Mistake 2: Stopping before gap becomes 1

This is wrong.

The final `gap = 1` pass is necessary.

Without it, the array may remain only partially sorted.

Important rule:

```
Shell Sort must end with gap = 1.
```

Mistake 3: Using `j--` instead of `j = j - gap`

Wrong:

```
j--;
```

Correct:

```
j = j - gap;
```

Why?

Because Shell Sort moves backward by gap distance, not by one position.

Mistake 4: Using `A[j + 1]` instead of `A[j + gap]`

Wrong:

```
A[j + 1] = A[j];
```

Correct:

```
A[j + gap] = A[j];
```

Why?

Because Shell Sort shifts values by gap distance.

Mistake 5: Forgetting to save `temp`

Wrong idea:

```
Shift values without saving the current value first.
```

Correct:

```
temp = A[i];
```

Why?

Because shifting may overwrite the value at `A[i]`.

Mistake 6: Assuming Shell Sort has one fixed time complexity

Shell Sort complexity depends on the gap sequence.

So do not memorize only one complexity without understanding this point.

Safe answer:

Shell Sort is usually faster than simple $O(n^2)$ sorts, but exact complexity depends on the gap sequence.

41. Shell Sort Quick Revision Table

Concept	Simple answer
What is Shell Sort?	Improved Insertion Sort
Main tool	Gap
First gap	Usually <code>n / 2</code>
Gap update	<code>gap = gap / 2</code>
Final gap	<code>1</code>
Movement style	Shift by gap distance
Code condition	<code>A[j] > temp</code>
Insert position	<code>A[j + gap]</code>
Extra memory	<code>O(1)</code>
Stable?	Usually no
Best memory trick	Big gaps first, gap 1 finishes

42. Final Algorithm Selection Guide

Now that you have completed all phases, you should know when to choose each sorting algorithm.

Situation	Good choice	Why
You are learning sorting for the first time	Bubble Sort	Very visual and easy to understand
You want to understand adjacent swaps	Bubble Sort	It compares neighbors repeatedly
Array is already sorted or nearly sorted	Insertion Sort	Very fast on nearly sorted data
You need an adaptive simple algorithm	Insertion Sort or Adaptive Bubble Sort	They can reduce work on sorted data

Situation	Good choice	Why
You are sorting a linked list	Insertion Sort	Inserting by changing links is suitable
You want fewer swaps	Selection Sort	It swaps only once per pass
You need the k largest values after k passes	Bubble Sort	Each pass fixes one largest value at the end
You need the k smallest values after k passes	Selection Sort	Each pass fixes one smallest value at the front
You want fast average in-place sorting	Quick Sort	Average time is $O(n \log n)$
You want guaranteed $O(n \log n)$ time	Merge Sort	Best, average, and worst are $O(n \log n)$
You need stable sorting with good time	Merge Sort	Can be stable if equal values are handled correctly
You have small non-negative integer values	Count Sort	Counting array is efficient
Values fit naturally into bins	Bin Sort	Uses bins or linked lists
You have large non-negative integers with digits	Radix Sort	Sorts digit by digit
You want to improve Insertion Sort without extra arrays	Shell Sort	Uses gaps and stays in-place

43. Comparison of All Main Algorithms

Algorithm	Main idea	Best case	Worst case	Extra space	Stable?	Adaptive?
Bubble Sort	Swap adjacent wrong pairs	$O(n)$ with flag	$O(n^2)$	$O(1)$	Yes	Yes with flag
Insertion Sort	Insert into sorted left part	$O(n)$	$O(n^2)$	$O(1)$	Yes	Yes
Selection Sort	Select minimum each pass	$O(n^2)$	$O(n^2)$	$O(1)$	Usually no	No

Algorithm	Main idea	Best case	Worst case	Extra space	Stable?	Adaptive?
Quick Sort	Partition around pivot	$O(n \log n)$	$O(n^2)$	Stack space	Usually no	No
Merge Sort	Split and merge sorted halves	$O(n \log n)$	$O(n \log n)$	$O(n)$	Can be yes	No
Count Sort	Count values	$O(n + \max)$	$O(n + \max)$	$O(\max)$	Not in simple version	No
Bin Sort	Put values into bins	Depends on range	Depends on range	Extra bins/nodes	Can preserve equal order with FIFO	No
Radix Sort	Sort digit by digit	$O(d \times n)$	$O(d \times n)$	Bins/extra space	Depends on stable bin handling	No
Shell Sort	Insertion Sort with gaps	Depends on gap sequence	Depends on gap sequence	$O(1)$	Usually no	Some adaptive behavior

44. Final Beginner Takeaway

Shell Sort is:

Insertion Sort + gaps

Insertion Sort moves values slowly:

one step at a time

Shell Sort moves values faster at first:

large gap jumps

Then it finishes with normal Insertion Sort:

```
gap = 1
```

The most important sentence is:

Shell Sort improves Insertion Sort by moving far-away elements closer to their correct positions before the final normal insertion pass.

Final memory trick:

```
Big gaps first.  
Small gaps later.  
Gap 1 finishes.
```

45. Very Short Exam-Style Summary

Shell Sort is an improved version of Insertion Sort. It uses gaps to compare and shift elements that are far apart. The gap usually starts at $n/2$ and keeps reducing by half until it becomes 1. Large gaps help move elements closer to their correct positions quickly. When the gap becomes 1, Shell Sort works like normal Insertion Sort, but the array is already nearly sorted. Shell Sort is in-place and uses $O(1)$ extra space. It is usually not stable, and its time complexity depends on the gap sequence.